

AI-Powered Research Assistant

Research, Evaluation & Prototype Implementation Report

Submitted for: AI Researcher / AI Innovation Engineer Evaluation Assignment

1. INTRODUCTION

The rapid growth of research publications, technical documentation, and enterprise knowledge repositories has significantly increased the complexity of information retrieval and literature analysis workflows. Traditional keyword-based search systems often fail to capture semantic relationships and contextual meaning within technical documents, making research workflows time-consuming and inefficient.

Recent advancements in Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) systems have enabled AI-powered workflows capable of semantic search, contextual understanding, and automated research assistance. By combining vector retrieval systems with generative AI models, modern research assistants can provide grounded responses using external document knowledge instead of relying solely on pretrained model memory.

This project focuses on the design and implementation of an AI-powered Research Assistant capable of:

- processing research PDFs,
- performing semantic retrieval,
- generating contextual answers,
- summarizing technical findings,
- and automating parts of the research workflow.

The implemented system uses:

- Gemini Flash Lite,
- LangChain,
- HuggingFace embeddings,
- FAISS vector retrieval,
- FastAPI backend services,
- and Streamlit frontend interfaces.

The primary objective of this project was to evaluate multiple AI tools and architectures, build a working prototype, and analyze the practical engineering tradeoffs involved in deploying an AI-powered research workflow system.

2. PROBLEM STATEMENT

Research professionals regularly work with large technical documents containing:

- dense terminology,
- citations,

- conceptual explanations,
- and domain-specific knowledge.

Traditional retrieval systems rely heavily on keyword matching and often struggle with:

- contextual understanding,
- semantic similarity,
- and natural language interaction.

As a result:

- literature review becomes time-intensive,
- information retrieval quality decreases,
- and researchers spend excessive time manually extracting insights from documents.

Standalone Large Language Models also introduce challenges such as:

- hallucinated responses,
- lack of grounding,
- and inability to reliably reference uploaded research documents.

To address these limitations, this project implements a Retrieval-Augmented Generation (RAG) architecture that combines:

- semantic embeddings,
- vector similarity search,
- and LLM-based reasoning
to generate grounded contextual responses from uploaded research PDFs.

The core problem addressed is:

“How can an AI-powered workflow system efficiently process technical documents and generate contextual research insights while remaining scalable, lightweight, and cost-efficient?”

3. EVALUATION METHODOLOGY

The evaluation focused on practical AI engineering considerations rather than benchmark-only comparisons. All technologies were evaluated within the same workflow:

1. PDF Upload
2. Text Extraction
3. Semantic Chunking
4. Embedding Generation
5. Vector Similarity Retrieval
6. Context Injection into LLM
7. AI-generated Response Generation

Metric	Description
Contextual Reasoning	Ability to generate grounded responses
Retrieval Quality	Semantic relevance of retrieved chunks

Integration Simplicity	Ease of workflow integration
Latency	Response generation speed
Cost Efficiency	Deployment affordability
Scalability	Production deployment suitability
Deployment Complexity	Infrastructure requirements

The system was tested using technical research PDFs and research-oriented queries such as:

- “What are the key findings in this document?”
- “Summarize the paper.”
- “Explain the role of foundation models.”
- “What limitations are discussed?”

4. AI TOOL & PLATFORM EVALUATION

4.1 Large Language Model Evaluation

Model	Strengths	Limitations
Gemini Flash Lite	Fast, low-cost, strong contextual reasoning	Free-tier quota limitations
GPT-4o	Excellent reasoning and summarization	Higher API cost
Claude 3.5 Sonnet	Strong long-context understanding	Limited free-tier access

Final Selection - Gemini Flash Lite

Gemini Flash Lite was selected because it provided the best balance between:

- reasoning quality,
- API accessibility,
- low latency,
- and operational cost.

Although GPT-4o produced slightly stronger reasoning outputs, Gemini was significantly more practical for lightweight deployment and rapid experimentation.

4.2 Retrieval System Evaluation

Vector Database	Advantages	Limitations
FAISS	Lightweight, fast local retrieval, free	Limited distributed scalability
Pinecone	Managed cloud infrastructure	Additional infrastructure complexity
ChromaDB	Simple persistent storage	Less optimized than FAISS

Final Selection - FAISS

FAISS was selected because:

- local retrieval was sufficient for prototype scope,

- deployment complexity remained minimal,
- and retrieval latency was consistently low.

Practical testing showed that retrieval quality depended more on:

- chunking strategy,
 - embedding quality,
 - and context filtering
- than on distributed vector infrastructure.

4.3 Embedding Model Evaluation

Embedding Model	Advantages	Limitations
MiniLM-L6-v2	Fast, lightweight, free	Slightly lower semantic precision
OpenAI Embeddings	High-quality embeddings	API cost and dependency

Final Selection - MiniLM-L6-v2

MiniLM embeddings provided strong semantic retrieval quality while allowing:

- local inference,
- low operational cost,
- and fast indexing performance.

4.4 PDF Processing Evaluation

Tool	Advantages	Limitations
PyPDFLoader	Lightweight and simple integration	Limited handling of complex layouts
LlamaParse	Better structured document parsing	API dependency
Unstructured.io	Enterprise-grade ingestion	Higher deployment complexity

Final Selection - PyPDFLoader

PyPDFLoader provided sufficient extraction quality for research PDFs while maintaining lightweight deployment and rapid integration into the RAG pipeline.

5. FINAL ARCHITECTURE DESIGN

The implemented architecture follows a modular Retrieval-Augmented Generation (RAG) workflow. Figure 1 illustrates the end-to-end Retrieval-Augmented Generation (RAG) workflow implemented in the proposed AI Research Assistant system.

Workflow Architecture

1. User uploads research PDF
2. PDF content is extracted
3. Document is chunked into semantic sections
4. Embeddings are generated
5. Embeddings are stored in FAISS
6. User query is embedded

7. Relevant chunks are retrieved
8. Retrieved context is injected into Gemini
9. Gemini generates grounded contextual responses

Final Technology Stack

Component	Selected Technology
LLM	Gemini Flash Lite
Orchestration	LangChain
Vector Database	FAISS
Embeddings	MiniLM-L6-v2
PDF Parsing	PyPDFLoader
Backend	FastAPI
Frontend	Streamlit

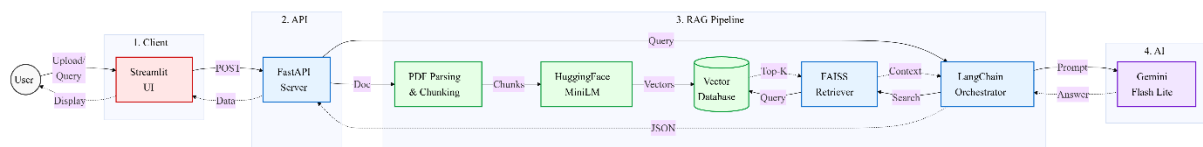


Figure 1. System Architecture

Multi-Agent Workflow

The system also includes a lightweight multi-agent workflow layer consisting of:

- Research Analysis Agent,
- Summarization Agent,
- and Recommendation Agent.

CrewAI orchestration was initially explored; however, lightweight sequential orchestration was ultimately selected due to:

- lower dependency complexity,
- improved implementation stability,
- and simpler integration during rapid prototyping.

6. Prototype Implementation

The implemented prototype supports:

- PDF upload,
- semantic retrieval,
- contextual question answering,
- and AI-generated research insights.

Backend Features

- FastAPI APIs
- RAG pipeline integration
- Semantic retrieval using FAISS
- Gemini-powered contextual generation

Frontend Features

- Streamlit-based UI
- PDF upload interface
- Natural language querying
- AI-generated response visualization

Practical Engineering Observations

During implementation:

- bibliography-heavy chunks occasionally reduced retrieval quality,
- chunk overlap improved contextual continuity,
- and lightweight embeddings proved sufficient for prototype-scale retrieval.

Simple retrieval filtering heuristics significantly improved response grounding and contextual accuracy.

7. Cost, Scalability & Limitations

Cost Analysis

Component	Cost
Gemini Flash Lite	Free-tier / low-cost
FAISS	Free
HuggingFace Embeddings	Free
FastAPI	Free
Streamlit	Free

The architecture was intentionally optimized for low-cost deployment and rapid experimentation.

Scalability Considerations

The system can be scaled using:

- Pinecone or Weaviate for distributed retrieval,
- cloud deployment infrastructure,
- persistent vector indexing,
- multi-user authentication,
- and advanced agent orchestration.

Limitations

Current limitations include:

- free-tier Gemini API quotas,
- prototype-scale vector storage,
- limited conversational memory,
- and simplified PDF parsing for complex layouts.

8. Conclusion

This project successfully demonstrates the design and implementation of an AI-powered Research Assistant using Retrieval-Augmented Generation (RAG) principles. The final system combines:

- semantic retrieval,
 - contextual LLM reasoning,
 - vector similarity search,
 - and lightweight AI workflow orchestration
- to automate parts of the research and literature analysis workflow.

The implemented architecture achieved a strong balance between:

- contextual performance,
- deployment simplicity,
- scalability,
- and operational cost.

Most importantly, the project demonstrates practical AI engineering capabilities involving:

- RAG systems,
- semantic search,
- LLM integration,
- API development,
- and workflow-oriented AI system design.